OFFLINE CONSTRUCTION KIT

Build SmartAttend with Codex

A self-contained, step-by-step Android application guide: project setup, prompts, Firebase, simulated attendance, teacher dashboard, testing and recovery.

14-day schedule | No ESP32 needed now | Copyable Codex prompts | Offline HTML

Read this first

One self-contained guide for finishing SmartAttend even if the ChatGPT or Codex conversation disappears.

Your goal: A real, runnable Android app + Firebase student login + clearly labeled simulated attendance + a basic teacher web dashboard. ESP32 hardware is **not required now**. Do not describe a simulation as real physical attendance.

Study plan

14 days • 5–7 focused hours/day
Beginner + AI-assisted. This is an aggressive demo deadline, not a guaranteed production delivery date.

Preserve your work

Keep Android source files locally, use Git checkpoints, update `PROJECT_STATUS.md`, and export checklist/notes from this HTML page.

What the supplied 11-page technical project guide specifies

- One classroom, Android-only student app, three ESP32 BLE beacons, Firebase, and teacher/admin web dashboard.
- Student app screens: Login/Registration, Home/Current Class, My Attendance, History, Profile.
- Data: students, subjects, timetable, sessions, attendance, devices, audit logs.
- Future BLE rule: multi-node RSSI measurements, stability/dwell time and exit timeout; classroom thresholds require real calibration.
- Example attendance policy: at least 75% Present, 50–74% Review, below 50% Absent; late arrival is a separate flag.

This handbook adapts that source design to an app-first, hardware-free 14-day build. Added engineering recommendations (Git, source abstraction, security, tests) are implementation guidance rather than statements from the original guide.

Build scope and architecture

Phase 1: no hardware

Student UI → Firebase login + timetable → SimulatedPresenceSource → attendance calculation → teacher dashboard.

Keep the source interface separate so `BlePresenceSource` can be added later. A teacher-controlled process should finalize official records; never trust arbitrary student-client writes as conclusive proof.

What you must NOT attempt during these 14 days

No face recognition, AI prediction, exact indoor positioning, iOS, multiple classrooms, Play Store release, or real BLE scanning without hardware. Avoid unsupported claims of anti-proxy accuracy.

Tools and links

Tool	Purpose	Get it
Android Studio	Create / run the Android app	https://developer.android.com/studio
Codex documentation	AI coding help and workspace guidance	https://developers.openai.com/codex/
Firebase Console	Authentication and Firestore	https://console.firebase.google.com/
VS Code	Optional editor for teacher dashboard	https://code.visualstudio.com/
Git	Local source history and checkpoints	https://git-scm.com/downloads
GitHub	Optional off-device code backup	https://github.com/
Arduino IDE (later)	ESP32 programming only after hardware arrives	https://www.arduino.cc/en/software

Part A — Start from zero on Windows

Do these steps before asking Codex to make large code changes.

1. Install Android Studio

Download from https://developer.android.com/studio, run the Windows installer, accept recommended SDK components, and wait for installation. A reasonably modern Windows laptop with enough disk space is needed.

> **Verify:** Android Studio opens to its welcome screen.

2. Create an Android project

Choose New Project > Empty Activity (Kotlin + Compose template). Name: SmartAttend. Package: com.smartattend.app. Minimum SDK: API 26. Save in C:\Projects\SmartAttend (or a folder you can find). Finish and wait for Gradle sync.

> **Verify:** MainActivity.kt and app/build.gradle.kts are present.

3. Run the untouched starter app

Connect an Android phone with USB debugging enabled; accept its debugging prompt. Or create an emulator in Android Studio Device Manager. Select device and press Run (green triangle).

> **Verify:** The starter screen opens on the phone/emulator.

4. Open the SAME folder in Codex

Desktop app: open/select the SmartAttend project folder. VS Code extension: File > Open Folder, choose SmartAttend, then open Codex. Codex web: connect a GitHub repository first; local instructions differ. Do not create a second unrelated project.

> **Verify:** Codex can inspect MainActivity.kt in the same folder Android Studio uses.

5. Run Prompt P0

Copy and paste P0 below into Codex. Permit only changes you understand. Ask Codex to tell you exactly what changed and whether Gradle actually built.

> **Verify:** AGENTS.md, PROJECT_STATUS.md, PROJECT_PLAN.md exist and build result is known.

6. Save your first Git checkpoint

In the project folder, initialize Git if needed and commit a first working state. Inspect .gitignore, especially local.properties, signing files, service credentials. Keep a second backup outside the laptop if possible.

> **Verify:** You can recover the working starter project even if later code fails.

> **Codex interface matters.** Desktop app or editor extension can usually work with the local folder directly. Codex web typically uses a connected repository. The key rule: Codex and Android Studio must refer to the SAME project, not separate copies.

Windows build and checkpoint commands

Open the terminal in the **SmartAttend project root** (where `gradlew.bat` exists):

```
.\gradlew.bat assembleDebug
```

After a successful debug build, the APK is usually at `app\build\outputs\apk\debug\app-debug.apk`. For version control, install Git and then use:

```
git init
git status
git add .
git commit -m "SmartAttend starter project"
```

Check `.gitignore` before adding files. Never commit passwords, private signing keys or service-account private keys. If Git asks for your identity, configure your name and email first.

Part B — Daily 14-day checklist

Check a day only after its verification condition is true. Progress is stored in this browser and can be exported.

- ☐ **Day 1 — Set up the project**

  Install Android Studio, create the Kotlin/Compose project, open the same folder in Codex, run the untouched starter app.

  > **Done when:** Starter app opens on Android phone or emulator.

☐ Day 2 — Make a safe starting point

Ask Codex to inspect files, write AGENTS.md and PROJECT_STATUS.md, run a build, and set up Git backup.

> **Done when:** Build succeeds and Git has a first checkpoint.

☐ Day 3 — Create the student interface

Implement the login UI and Home screen with clearly labeled demo data.

> **Done when:** App launches with functional login demo and Home screen.

☐ Day 4 — Add navigation and screens

Implement My Attendance, History, Timetable, Profile, and bottom navigation.

> **Done when:** All screens open without crashing.

☐ Day 5 — Model local data

Create mock students, subjects, sessions and records; add empty/loading/error UI states.

> **Done when:** UI changes correctly when the mock data changes.

☐ Day 6 — Prepare Firebase manually

Create Firebase project, Android app registration, auth, Firestore, and place google-services.json.

> **Done when:** Firebase configuration present; no credentials committed.

☐ Day 7 — Integrate student sign-in

Have Codex connect Firebase Authentication, profiles and safe student-only reads.

> **Done when:** Test user signs in and accesses only own profile.

☐ Day 8 — Add timetable and sessions

Store timetable, enrollment, and active session records; handle no-class and wrong-batch cases.

> **Done when:** App shows correct active class or rejection.

☐ Day 9 — Build attendance simulation

Create SimulatedPresenceSource, entry/exit controls, time clock and percentage calculation.

> **Done when:** Simulated presence works and is visibly labeled DEMO.

☐ Day 10 — Test attendance logic

Unit-test boundary cases, repeated entry, early exit, late arrival, wrong batch and session end.

> **Done when:** Critical tests pass; no duplicate time counting.

☐ Day 11 — Create teacher web dashboard

Build basic teacher sign-in, sessions table, and live attendance list in dashboard/ folder.

> **Done when:** Teacher can sign in and view a class.

☐ **Day 12 — Secure teacher actions and export**

Add controlled start/end class, corrections with audit log and basic CSV export.

> **Done when:** Teacher workflow works; students cannot edit final results.

☐ **Day 13 — Run the full demo and fix errors**

Demonstrate teacher start -> demo entry -> exit -> calculated record -> teacher view.

> **Done when:** End-to-end walkthrough succeeds with two accounts.

☐ **Day 14 — Produce the APK and submission pack**

Generate debug APK; save source ZIP, README, screenshots, limits, test notes and viva points.

> **Done when:** APK, source backup, test evidence and presentation ready.

Part C — Full copy/paste Codex prompts

Paste ONE prompt at a time. Test and save a checkpoint before moving to the next prompt. They are ordered to match your 14-day plan.

P0 **Codex project rules / first inspection** Day 1–2

```
You are the lead engineer and beginner-friendly instructor for SmartAttend, my final-year
project.

CONTEXT
- I am a complete beginner with 14 days.
- I have a valid Android Studio Kotlin + Jetpack Compose project open in this workspace.
- I do NOT have ESP32 hardware yet.
- Ultimate design: 3 ESP32 BLE beacons -> Android student app -> Firebase -> teacher web
dashboard.
- Today is only project inspection and safe planning.

TASKS
1. Inspect the existing project and report the module, Kotlin/Gradle/SDK setup and existing
files.
2. Do not delete, recreate or overwrite a functioning Android project.
3. Create AGENTS.md with rules: work one milestone at a time; preserve working features; use
maintainable beginner-friendly code; don't pretend unrun tests passed; label simulated data;
```

```
never expose secrets; prefer incremental fixes.
4. Create PROJECT_STATUS.md with Completed, Current task, Tests run, Known issues, Next
action.
5. Create PROJECT_PLAN.md for Android UI, Firebase, simulated attendance, teacher dashboard,
QA, then future BLE.
6. Recommend a simple PresenceSource interface that can later have SimulatedPresenceSource
and BlePresenceSource implementations; do not implement BLE now.
7. Run the existing debug build if tools allow; tell me the exact command and result.
8. Explain in simple terms what I should do manually.

STOP after inspection/planning. Report files changed, verified results, blockers and the next
tiny step.
```

P1 First student screen + working app Day 3

```
Build the first real SmartAttend Android screen in this existing Kotlin/Jetpack Compose
project.

GOAL
A polished student demo login and Home screen that compiles and runs. Use Material 3.

STYLE
Primary #1769E0; navy #123563; white/light gray; good contrast; simple rounded cards; clean
college-app design. Avoid excessive animation.

FUNCTIONS
- Demo Login with student ID or email and password fields; show DEMO MODE prominently.
- Use local fake student data for now; do not claim this is secure authentication.
- Home shows student's name, current lecture card, room, session status, and example
attendance overview.
- Show loading/empty state where relevant.

REQUIREMENTS
- Do not add Firebase or Bluetooth yet.
- Make code changes directly in this workspace, do not only print snippets.
- Keep code small, understandable and easily testable.
- Explain exact files changed and how I run the app.
- Run a Gradle debug build if possible; if not, explain what couldn't be tested.
- Update PROJECT_STATUS.md and stop.
```

P2 Navigation and five student screens Days 4–5

```
Extend SmartAttend using the existing Kotlin/Jetpack Compose project.

IMPLEMENT THESE STUDENT SCREENS
1. Home/Current Class: lecture, room, class time, detection status.
2. My Attendance: subject-wise attended/total classes, percentages and shortage warning.
3. History: date, subject, Present/Late/Review/Absent status.
4. Timetable: today's scheduled lectures and active session indicator.
5. Profile: student name, roll number, department, batch and future device placeholder.

Add functional navigation, consistent styles, local sample-data repository and helpful empty/
error UI. Demo data must be visibly labeled.
```

```
Do not add Firebase or BLE. Do not accidentally break the working login screen.
Run build checks, explain exactly how to tap through every screen, update PROJECT_STATUS.md,
stop.
```

P3 Firebase readiness and rules planning Day 6

```
Prepare the existing SmartAttend Android app for Firebase integration, but do not invent
credentials or assume I have finished Firebase Console setup.

First inspect Gradle files and check whether app/google-services.json exists.

I will manually create the Firebase project, register Android package com.smartattend.app,
download google-services.json, enable email/password authentication, and create Firestore.

TASKS
- Tell me which Firebase Console steps are still missing.
- Add compatible Firebase Android plugins/dependencies only when configuration is present or
when it is safe to do so.
- Use currently supported Firebase SDK artifacts and the BoM; avoid obsolete KTX artifacts.
- Prepare a concise data model for users, subjects, enrollments, timetable, sessions,
presenceEvents, attendance and auditLogs.
- Draft deny-by-default security rules: student only own data; no student write to finalized
attendance; teacher-only authorized operations; no self-assignment of role.
- Explain a safe way to manually bootstrap the first teacher account in Firebase Console.
- Do not put passwords or service-account private keys in source code.
- Run available build checks, update PROJECT_STATUS.md and stop.
```

P4 Firebase student login and profile Day 7

```
Connect SmartAttend to the Firebase project already configured in this workspace.

IMPLEMENT
- Email/password student authentication with Firebase Authentication.
- Login/logout and persistent signed-in state.
- Firestore profile reads tied to Firebase Auth UID.
- Loading, success, no-profile, and meaningful error states.
- Students must not read another student's profile or attendance.
- Keep a clearly separated mock/demo data mode for offline UI work.

SECURITY
- Do not let users choose their own teacher/admin role.
- Include/adjust Firestore rules and a clear console deployment checklist.
- Do not allow student writes to final attendance records.
- Explain any required manual Firestore test-user and profile setup.

Keep the app building, add appropriate tests if practical, update PROJECT_STATUS.md, and tell
me how to verify using two test student accounts. Stop after this milestone.
```

P5 Timetable and active class validation Day 8

```
Implement the data and app logic for SmartAttend active class sessions.

Data concepts: student, batch/enrollment, subject, room, timetable, teacher, session start/
end, session status.

FEATURES
- Show today's timetable and the current active class on Home.
- Check whether signed-in student is enrolled in the active batch/subject.
- Distinguish No class, Upcoming class, Active eligible class and Active ineligible class.
- Do not create any attendance record from merely opening the app.
- Use consistent UTC timestamps for storage and display campus-local times clearly.
- Validate user/session relationships with appropriate Firestore rules or trusted teacher-
controlled records.
- Provide test sample records with instructions for entering them safely.

Do not implement real BLE scanning. Run tests/build, explain the verification steps and
update PROJECT_STATUS.md. Stop.
```

P6 Simulated attendance engine Days 9–10

```
Implement a clearly labeled SIMULATION ONLY attendance engine in the existing SmartAttend
Android app. I do not own any ESP32 boards yet.

ARCHITECTURE
- PresenceSource abstraction that emits presence state/events.
- SimulatedPresenceSource with manual Enter Classroom and Exit Classroom buttons.
- Keep a future BlePresenceSource placeholder/integration note separate; no fake BLE
readings.
- AttendanceCalculator and a testable clock abstraction.

LOGIC
1. Require a signed-in student who is enrolled in an active subject/batch/session.
2. On demo Enter, record event and start accumulating duration.
3. On demo Exit, close the current interval.
4. Re-entry continues accumulating time without duplicate counting.
5. At session end, close any open interval and compute percentage = confirmed minutes /
scheduled or configured lecture minutes * 100.
6. Example policy: >=75% Present, 50% to <75% Review, <50% Absent. Track Late separately
using a configurable grace period.
7. Reject inactive/wrong-batch/duplicate events.
8. Use a controllable fake clock for fast testing without waiting an hour.
9. Label ALL simulated records and screens DEMO / NOT VERIFIED BY BLE.
10. Separate untrusted student-reported presence evidence from teacher-authorized final
records.

TESTS
100%, 75%, 74.9%, 50%, 49.9%, re-entry, repeat enter/exit, late arrival, early exit, wrong
enrollment, inactive session, session end with open interval.

Do not claim proxy prevention or secure physical presence. Run real tests/build where
possible; update PROJECT_STATUS.md and stop.
```

P7 Teacher dashboard Days 11–12

```
Build a basic SmartAttend teacher/admin web dashboard in the existing repository folder
teacher-dashboard/ using HTML, CSS, JavaScript and Firebase Web SDK.

REQUIREMENTS
- Reuse the app's existing Firestore collection names and document structure.
- Teacher sign-in via Firebase Auth.
- Responsive clean navy/blue UI.
- Today's timetable and class sessions.
- Start/stop a class session where the authenticated account is authorized.
- Live attendance/demo evidence table.
- Finalized attendance table with Present/Late/Review/Absent.
- Teacher correction requires written reason and writes an audit record.
- CSV export for class attendance.

SECURITY
- Teacher roles must be assigned outside student self-service.
- Enforce rights in Firestore rules/trusted backend, not only by hiding buttons.
- Prevent students from directly setting attendance statuses or teacher permissions.
- Never embed service-account private keys.
- Clearly indicate demo/simulated presence.

Scope: one classroom, one teacher, a few demo student accounts. No complex analytics or
production deployment.

Tell me exactly how to run the web dashboard locally, finish required Firebase console steps,
test teacher vs student permissions, and run/build checks. Update PROJECT_STATUS.md and stop.
```

P8 End-to-end demo rehearsal Day 13

```
Audit and test the existing SmartAttend prototype end to end. Do not start redesigning the
UI.

DEMO FLOW
1. Teacher signs in and starts a class.
2. Eligible student signs in.
3. Student sees active eligible session.
4. Student explicitly uses SIMULATED entry and exit.
5. App computes presence duration and percentage.
6. Authorized teacher sees evidence and final attendance result.
7. Teacher makes a justified correction and audit log records the action.
8. Student sees updated finalized status.

NEGATIVE CASES
- Wrong-batch student
- Inactive session
- Student tries to edit own finalized attendance
- Student attempts teacher operation
- Double entry and repeated exit
- Internet unavailable or Firestore operation fails

For each test report PASS, FAIL or NOT RUN and include evidence (command/result or human
manual verification needed). Fix only confirmed bugs. Update PROJECT_STATUS.md and write
TEST_REPORT.md. Stop.
```

P9 APK, backup, documentation and viva

Day 14

```
Prepare this SmartAttend repository for final-year college demonstration.

DELIVERABLES
1. Verify Gradle assembleDebug, provide precise APK path only if generated.
2. Write README.md: project purpose, architecture, stack, Windows setup, run steps, how DEMO
mode works, Firebase setup, teacher dashboard launch.
3. Write TEST_REPORT.md with actual results, unrun tests and limitations.
4. Write DEMO_SCRIPT.md with a 5-minute walkthrough.
5. Write VIVA_NOTES.md with BLE/RSSI, why 3 ESP32 nodes, Firebase, presence duration,
security and limitations.
6. Include notes on how to add real ESP32 hardware later.
7. Produce a list of files to back up and any environment-specific files that need
reconfiguration.
8. Update PROJECT_STATUS.md so a new Codex chat can resume.

Do not claim real BLE, GPS accuracy, fraud-proof attendance or full production security. Run
checks and list exact success/failure. Stop.
```

P10 Fix an error without damaging the app

Whenever something breaks

```
You are fixing an error in my existing SmartAttend repository. I am a programming beginner.

ERROR / SYMPTOM:
[PASTE THE FULL ERROR OR DESCRIBE WHAT HAPPENED]

WHAT I DID BEFORE IT FAILED:
[EXPLAIN LAST STEP]

TASK
1. Inspect the related source and build files before changing anything.
2. Explain the cause simply.
3. Apply the smallest safe change that addresses the observed error.
4. Avoid rewriting working screens or architecture.
5. Run the relevant build/test if available; be honest about what was not run.
6. Report which files changed and how I can verify the fix.
7. Update PROJECT_STATUS.md.

Do not add unrelated new features.
```

P11 Resume development in a fresh Codex chat

Use if browser/Codex session disappears

```
We are resuming SmartAttend after losing prior chat context. Do not assume anything from
memory.

PROJECT:
Android Kotlin/Jetpack Compose, Firebase, a separate teacher dashboard, simulated attendance
only for now. No ESP32 boards yet.

FIRST ACTIONS
```

```
1. Read AGENTS.md, PROJECT_STATUS.md, PROJECT_PLAN.md and README.md if available.
2. Inspect Git status and the relevant current files.
3. Tell me what already works, which tests were actually run, unresolved errors and the most
important next milestone.
4. Do NOT restart the project, change dependencies without reason, or recreate features
already working.
5. Suggest ONE small next coding task and ask for my approval before editing.

Keep every response beginner-friendly and update PROJECT_STATUS.md after work.
```

P12 **Later: connect actual ESP32 BLE boards** Only after buying hardware

```
SmartAttend already has a tested simulated PresenceSource. Now integrate 3 physical ESP32 BLE
beacons without rewriting other attendance logic.

FIRST: inspect existing code, ask for exact ESP32 board model and Android phone version.

THEN IN SMALL STEPS:
1. Provide compatible Arduino ESP32 BLE beacon code for one node, with unique classroom/node
identifiers.
2. Help me flash and detect one ESP32 via a phone scanner.
3. Build the Android BLE scanning source with Android-version-appropriate permissions and
lifecycle handling.
4. Show RSSI from one node, then all three.
5. Add filtering, an initial configurable 2-of-3 consistency/dwell rule, and a signal-loss
timeout.
6. Calibrate inside seats, outside walls, doorway and corridor with multiple phones.
7. Replace demo PresenceSource only in an explicitly selected real-hardware mode.
8. Preserve demo mode for troubleshooting, and never label it hardware verified.
9. Document background scanning restrictions, device variability, spoofing risk and absence
of exact indoor location.
10. Test, record measured results and keep teacher correction path.

Do not claim perfect anti-proxy, exact triangulation or secure presence solely from RSSI.
Start with inspecting the current repository and giving me the first physical setup step, not
a huge rewrite.
```

Part D — Firebase manual checklist

These steps are performed in your own Firebase Console. Codex can write code, but it cannot invent your cloud configuration.

1. Open console.firebase.google.com and create a Firebase project named SmartAttend.
2. Add an Android app with `com.smartattend.app` (must exactly match your app ID).
3. Download `google-services.json` and put it inside `SmartAttend/app/`.
4. Enable Email/Password in Authentication → Sign-in method.
5. In Cloud Firestore, create the database in a suitable region. Deploy restrictive security rules; do not leave a public test database.
6. Create at least two student test accounts and one teacher test account. Teacher permissions must be assigned by a trusted admin, never through an ordinary student registration form.

7. Use the Firebase Android setup guide to check compatible plugins and SDK modules. Do not put a service-account JSON key in the Android or web app.
8. Use P3 and P4 for integration, test separate identities, and verify that students cannot edit finalized records.

> **Security limitation:** BLE presence data sent by a student's phone is untrusted client input. This demo should never be represented as tamper-proof official attendance. Classroom policy and backend validation are required for a real deployment.

Part E — Final acceptance checklist

Use actual test outcomes, not guesses. Mark only what you have personally verified.

- ☐ Starter Kotlin app builds and launches
- ☐ Demo login and student navigation work
- ☐ All mock screens clearly show demo data
- ☐ Firebase student login works and logout works
- ☐ Another student cannot view private profile data
- ☐ Timetable correctly shows an active class
- ☐ Wrong-batch student is denied attendance flow
- ☐ Simulated Enter and Exit controls work
- ☐ No double-counted minutes or duplicate results
- ☐ 75%, 50% and below-50% boundary rules are tested
- ☐ Teacher starts and ends a session
- ☐ Teacher can view attendance and correct with audit reason
- ☐ Student cannot edit a finalized attendance status
- ☐ End-to-end demo works with test accounts
- ☐ Debug APK generated and source backed up
- ☐ Known limitations and future BLE phase documented

Practice a 5-minute viva demonstration

1. Say the problem: one-time check-ins do not prove continued presence.
2. Show the student screens and Firebase login.
3. Start a session as teacher in the web dashboard.
4. Simulate a student's entry and exit, calculate attendance, show it on dashboard.
5. Demonstrate wrong-batch rejection and teacher correction audit log.
6. State clearly: ESP32 integration is future phase; simulated signals are NOT live BLE.
7. Explain expected multi-node BLE behavior and its main limitation: RSSI variation and wall penetration.

Part F — Troubleshooting and recovery

For any coding issue, give Codex the full error and paste P10. Do not ask it to rewrite everything.

Codex says it cannot find project files

It is likely working in the wrong folder. In Android Studio use File > Open to locate the project; then open that exact directory in Codex. Check that settings.gradle.kts is visible.

Gradle sync or build fails

Copy the full first error, including lines above it, and use Prompt P10. Check internet connectivity for downloading dependencies, installed SDK, JDK configuration and Gradle compatibility. Do not blindly replace version numbers.

App does not appear on phone

Enable Developer Options and USB Debugging, confirm phone authorizes this PC, try a data-capable USB cable, select the actual device and hit Run. Alternatively use an emulator.

The app looks good but nothing changes

Check whether you are using demo/mock data. Verify buttons have real callbacks, view models observe state, and there is no hardcoded UI-only fake logic.

Firebase login fails

Check email/password provider enabled, the user exists, network is working, google-services.json matches your package/project, and the full Firebase error code. Never share passwords in prompts.

Firestore permission denied

Likely security rules or document UID mismatch. Do not change rules to allow read/write for everyone; inspect Auth UID, document paths and intended roles.

Teacher has no access

Check the teacher account has been granted a trusted role by an administrator; never add a client-side “make me teacher” button.

Simulated attendance duplicates minutes

Check overlapping intervals, repeated Enter events and session-end closure. Use test clock and unit tests; run Prompt P6/P10.

App lost after browser tab closed

Local files remain in your project folder, and this guide stays available offline. Open the same folder, then paste P11 into a new Codex chat.

Need to pause a project for a day

Ask Codex to update PROJECT_STATUS.md and commit or back up files. Export progress/notes from the HTML handbook as a separate .json backup.

Minimal recovery checklist after a lost chat

1. Find your local `SmartAttend` project folder.
2. Open it in Android Studio and in Codex.
3. Open `PROJECT_STATUS.md` to see the current milestone.
4. Run `.\gradlew.bat assembleDebug` or the Android Studio Build action.
5. Paste P11 into a fresh Codex chat.
6. Restore progress notes from a JSON export below, if needed.

Part G — Save progress and protect your files

The HTML works offline, including the checklist and prompts. Progress is saved to this browser unless you export it.

Project notes — edit anytime

Write your most recent Codex result, blockers, Firebase project status, last Git commit, and the next action. Notes are stored locally on this browser.

Export after each study session. Local browser storage can disappear if your browser data is cleared or you move to another computer.

Minimum offline backup folder: keep (1) the SmartAttend source directory or Git repository, (2) this HTML guide, (3) the PDF guide, (4) `PROJECT_STATUS.md`, (5) your exported progress JSON, and (6) a copy of the debug APK after it exists. Store the backup in cloud storage or an external drive too.

Part H — Connecting ESP32 later

This is Phase 2 after the app-only demo is working and you obtain three compatible BLE ESP32 boards.

1. Program and verify one ESP32 BLE advertiser using Arduino IDE.
2. Use an Android BLE scanner to confirm the identifier and RSSI.
3. Implement `BlePresenceSource` for Android with the correct permissions.
4. Add the remaining two beacons and record separate node RSSI values.
5. Use filtering and initial two-of-three stable presence logic with dwell/exit timeout; calibrate at front/back seats, door, corridor and behind walls.
6. Record measured false-positive and false-negative rates; do not claim exact indoor triangulation.
7. Test background behavior on your phone with battery optimization and Android foreground-service limitations in mind.
8. Maintain teacher corrections and document proxy-phone, spoofing and client-trust limitations.

When the hardware arrives, use prompt P12. The source guide gives illustrative 10–15-second confirmation windows and example RSSI values, but they are **not** universal thresholds.

SmartAttend app-first recovery handbook
Based on the user-supplied 11-page SmartAttend Complete Technical Project Guide, adapted to an Android-first, no-hardware, 14-day beginner development plan. All sample records and mockups must be labeled DEMO. This HTML is self-contained and works offline; the external links require internet.